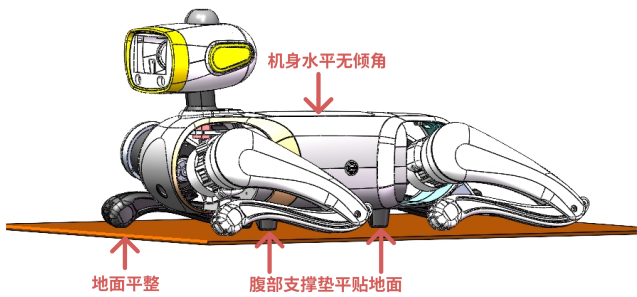


基于Vbot的二次开发

1. 开/关机、充电

- **开机：**电源键位于机身左侧，关机状态下，长按电源键2.5秒，电源键闪烁一次，白色耳灯亮起流水灯，即示意已通电。
 - **⚠️注意事项：**
 - **机身摆放需平整：**请确保开机前机身水平趴在**平整地面**上（如下图），机器人腹部支撑垫（共四个）需**平贴地面**，机身**水平无倾斜**趴在地面，机器人小腿呈收起状态（如下图），四个膝关节以及足端平放在地面上，确保机器人大腿和小腿都没被机身压住。



机身摆放示意图

- **关机：**开机状态下，长按电源键10秒，即启动关机程序，机器人会自动趴下并关闭。
 - **⚠️注意事项：**
 - **先趴下再关机：**关机时，应当先让机器人趴下，避免机器人断电摔倒，当机器人在运动或站立时，请避免直接关闭电源。
- **电源灯状态说明：**

Action/电量	0-10%	10-25%	25-50%	50-75%	75-100%
短按一次	红灯快闪5次（速度2次/秒）	红灯亮（2.5秒）	黄灯亮（2.5秒）	绿灯亮（2.5秒）	蓝灯亮（2.5秒）
开机时	红灯闪（2次/秒）	红灯亮	黄灯亮	绿灯亮	蓝灯亮
充电时	红灯闪（1次/秒）	红灯闪（1次/秒）	黄灯闪（1次/2秒）	绿灯闪（1次/2秒）	蓝灯闪（1次/2秒）

- **充电：**
 - 随整机配备一个140w的Type-C充电器，请使用该充电器为机器人充电；
 - 将Type-C连接线按图示插入机器人背部的对应接口（在盖板下方）并将电源适配器接通电源。

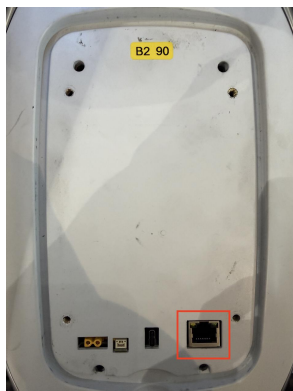


充电方式示意

- ⚠️ **注意事项**
 - **可能偶现无法充电的情况：**关机重启后重试可以解决；
 - **避免电量完全耗尽才充电：**可能会出现充电缓慢的情况，如长时间充不进去电，请与工作人员联系。
- ⚠️ **日常使用注意事项**
 - **避免沾水：**避免机器人沾水；
 - **避免夹手：**避免让大人或小孩接触机器人的移动部件（腿，脖子，前后甲中间空隙等）；
 - **避免烫伤：**机器人长时间工作后，大腿内外侧可能温度较高，避免直接用手触摸；
 - **保持对机器人和周边环境监护：**使用各功能时，保持观察机器人周边环境和机器人本体情况，避免碰撞等危险情况；
 - **电机过载或过热时，**机器人可能表现异常。机器人有任何异常状态时，请优先执行下文中的「急停」服务并关机重启；
 - **避免长时间高强度使用，**让机器人适度休息。

2. 快速开始

用网线的一端连接Vbot网口，另一端连接用户电脑，并开启用户电脑的USB Ethernet后进行配置。Vbot的IP地址为192.168.125.2，故需将用户电脑USB Ethernet地址设置为与Vbot同一网段，如在Address中输入192.168.125.10（“10”可以改成其他）。子网掩码输入255.255.255.0

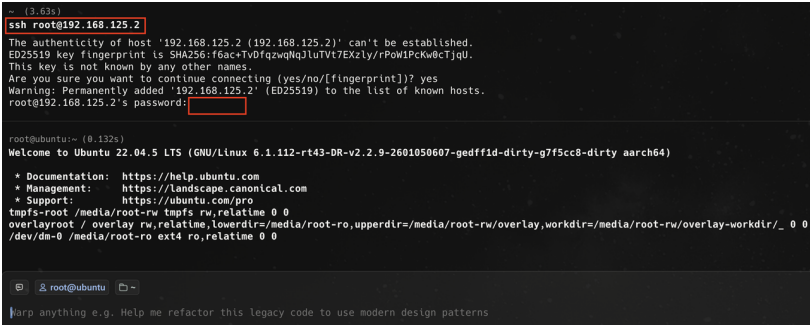


Vbot网口

在用户电脑终端执行ssh命令即可登陆Vbot：

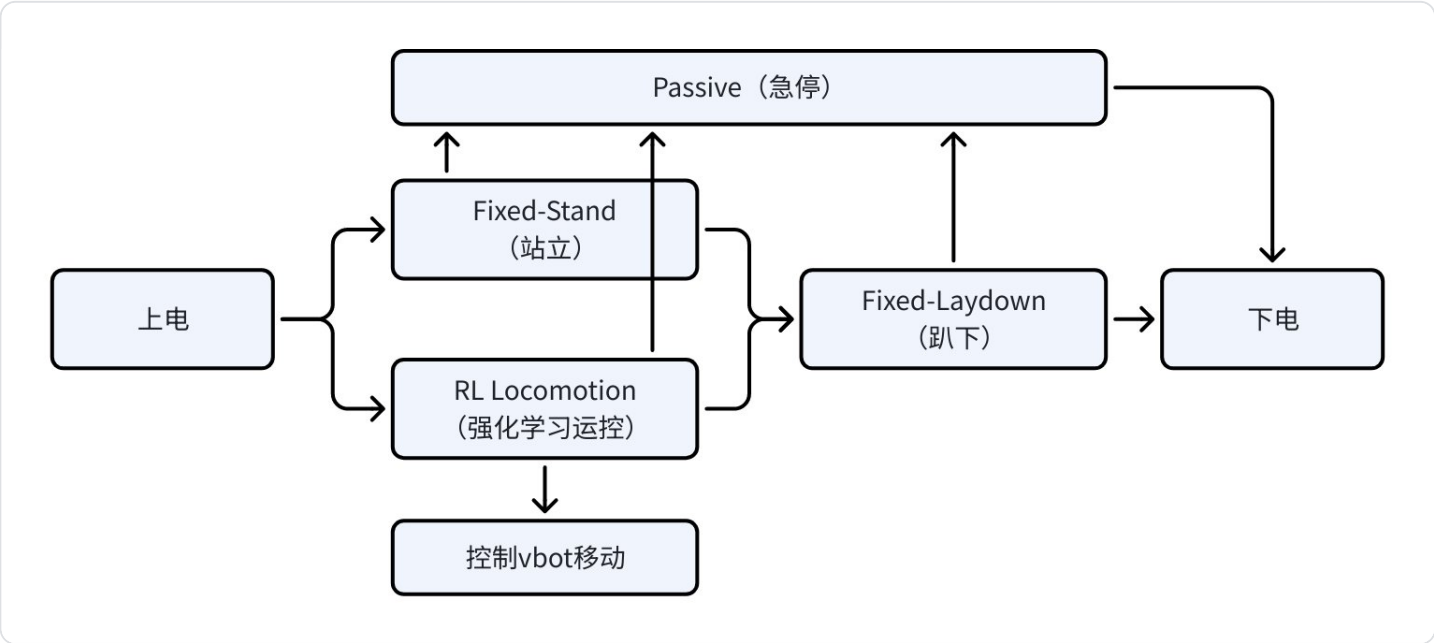
代码块

```
1 ssh root@192.168.125.2
2 # 密码：root
```



在用户电脑终端通过ssh命令登陆Vbot

3. 状态转换示意图



3.1 Passive（急停）

代码块

```
1 source /app/script/env.sh
2 ros2 service call /sm/action/lowlevel software_msgs/srv/LowlevelAction "
  {target_state: 1, mode: 0}"
3
4 # requester: making request:
  software_msgs.srv.LowlevelAction_Request(target_state=1, mode=0,
  pre_check=False, action_path='')
5 # response:
```

```
6  # software_msgs.srv.LowlevelAction_Response(success=True, message='Executed
    IMMEDIATE action: PASSIVE', error_code=0)
```

3.2 Fixed-stand（站立）

代码块

```
1  source /app/script/env.sh
2  ros2 service call /locomotion/set_run_mode function_msgs/srv/SetRunMode "
    {target_state: 1, mode: 1}"
3
4  # requester: making request:
    function_msgs.srv.SetRunMode_Request(target_state=1, mode=1, pre_check=False)
5  # response:
6  # function_msgs.srv.SetRunMode_Response(success=True, message='Run mode set
    successfully')
```

3.3 Fixed-laydown（趴下）

代码块

```
1  source /app/script/env.sh
2  ros2 service call /sm/action/lowlevel software_msgs/srv/LowlevelAction "
    {target_state: 1, mode: 2}"
3
4  # requester: making request:
    software_msgs.srv.LowlevelAction_Request(target_state=1, mode=2,
    pre_check=False, action_path='')
5  # response:
6  # software_msgs.srv.LowlevelAction_Response(success=True, message='Transition
    requested: FIXED_LAYDOWN', error_code=0)
```

3.4 RL Locomotion（强化学习运控）

3.4.1 进入RL运控

代码块

```
1  source /app/script/env.sh
2  ros2 service call /locomotion/set_run_mode function_msgs/srv/SetRunMode "
    {target_state: 1, mode: 2}"
3  # requester: making request:
    function_msgs.srv.SetRunMode_Request(target_state=1, mode=2, pre_check=False)
```

```
4 # response:
5 # function_msgs.srv.SetRunMode_Response(success=True, message='Run mode set successfully')
```

3.4.2 控制vbot移动

代码块

```
1 source /app/script/env.sh
2 ros2 topic pub /vel_cmd geometry_msgs/msg/Twist "{linear: {x: a, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: b}}"
3
4 # 以0.3m/s的速度往前行进
5 # ros2 topic pub /vel_cmd geometry_msgs/msg/Twist "{linear: {x: 0.3, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
6
7 # 以0.6rad/s的速度向左转
8 # ros2 topic pub /vel_cmd geometry_msgs/msg/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.6}}"
```

名称	含义	单位	取值范围
a	线速度	m/s	前进 [0.3, 1.5] 后退 [-0.3, -1.5]
b	角速度	rad/s	机身向左转 [0.5, 3.0] 机身向右转 [-0.5, -3.0]

4. 详细接口说明

4.1 功能说明

Dev Edition包含完整的传感器接口，适用于二次开发。

平台配置

平台	Function Groups	说明
S100	control, sensor, util	运动控制、传感器
X5	control, sensor, util	外设服务 (display/emotion, light, gnss)

功能可用性汇总

功能	状态	位置
✔ 运动控制 (lowlevel_service)	可用	S100 control FG
✔ 双目相机 (stereo)	可用	S100/X5 sensor FG
✔ 激光雷达 (lidar)	可用	S100 sensor FG
✔ 表情显示 (display/emotion)	可用	X5 control FG
✔ 灯光控制 (light)	可用	X5 control FG
✔ 触摸感应 (touch)	可用	S100 control FG
✔ 风扇控制 (fan)	可用	S100 control FG
✔ GNSS定位	可用	X5 control FG

4.2 运动控制 (lowlevel_service)

底层服务，负责电机驱动、运动状态反馈、头部控制等核心功能。

4.2.1 Topics

4.2.1.1 /rt/lowstate

类型: lowlevel_msg/LowState

描述: 底层状态实时数据流（高频发布，~500Hz）

IDL 定义:

代码块

```
1  # 核心传感器数据
2  IMUState imu_state           # IMU状态数据
3  MotorCmd[14] relative_motor_cmd  # 相对电机指令(12腿+2头)
4  MotorState[14] motor_state     # 电机状态反馈(12腿+2头)
5
6  # 状态同步信息
7  uint16 state_id              # 状态ID
8  uint16 input_cmd_id          # 输入指令ID
```

IMUState 子消息:

1	float32[4] quaternion	# 四元数
2	float32[3] gyroscope	# 陀螺仪
3	float32[3] accelerometer	# 加速度计
4	float32[3] rpy	# 欧拉角(RPY)
5	int8 temperature	# 温度

MotorCmd 子消息:

代码块

1	# 控制模式枚举	
2	uint8 KEYFRAME_INTERPOLATION = 0	# 关键帧插值模式
3	uint8 STREAMING_FILE = 1	# 文件流模式
4	uint8 RL_REALTIME = 2	# 强化学习实时模式
5		
6	# 电机控制参数	
7	uint8 mode	# 控制模式
8	float32 q	# 关节位置(角度)
9	float32 dq	# 关节速度
10	float32 tau	# 关节力矩(前馈)
11	float32 kp	# 位置增益(刚度)
12	float32 kd	# 速度增益(阻尼)
13	uint32[3] reserve	# 保留字段

MotorState 子消息:

代码块

1	uint8 mode	# 电机模式
2	float32 q	# 关节位置
3	float32 dq	# 关节速度
4	float32 ddq	# 关节加速度
5	float32 tau_est	# 估计力矩
6	float32 q_raw	# 原始关节位置
7	float32 dq_raw	# 原始关节速度
8	float32 ddq_raw	# 原始关节加速度
9	int8 temperature	# 温度
10	uint32 lost	# 丢包计数
11	uint32 accumulated_running_sec	# 累计运行时间(秒)
12	uint32[3] thermal_integral	# 热积分(过热保护)
13	uint32[2] reserve	# 保留字段

ROS2 CLI示例:

```

1  # 订阅底层状态
2  ros2 topic echo /rt/lowstate
3
4  # 查看发布频率
5  ros2 topic hz /rt/lowstate

```

4.2.1.2 /lowlevel_feedback

类型: `lowlevel_msg/LowlevelFeedback`

描述: 动作执行状态反馈

IDL 定义:

代码块

```

1  # 标准ROS2消息头
2  std_msgs/Header header          # 消息头
3
4  # 动作信息
5  string action_name              # 动作名称(如"FIXED_STAND", "SHAKEHAND")
6
7  # 状态常量定义
8  uint8 STARTED=0                 # 动作已开始
9  uint8 RUNNING=1                 # 动作运行中
10 uint8 SUCCESS=2                 # 动作成功完成
11 uint8 ERROR=3                   # 动作失败
12 uint8 INTERRUPTED=4             # 动作被中断
13
14 # 状态信息
15 uint8 status                     # 当前动作状态
16 string status_message           # 状态信息或错误描述
17
18 # 进度信息
19 float32 progress_percentage     # 当前进度(0.0-100.0)
20 float32 execution_duration      # 当前执行时长(秒)

```

ROS2 CLI 示例:

代码块

```

1  ros2 topic echo /lowlevel_feedback

```

4.2.1.3 /bms_state

类型: `lowlevel_msg/BmsState`

描述: 电池管理系统状态（独立Topic，更高频率）

IDL 定义:

代码块

```
1  # 系统控制
2  bool request_shutdown          # 是否请求关机
3
4  # 电池状态
5  uint32 voltage_mv              # 总电压 (mV)
6  int32 current_ma               # 电流 (mA)
7  float32 soc_percent            # 剩余电量百分比 (0-100)
8  float32 soh_percent            # 健康状态百分比 (0-100)
9  uint32 remaining_capacity_mah  # 剩余容量 (mAh)
10
11 # 循环与容量
12 uint16 cycles                   # 充放电循环次数
13 uint32 full_charge_capacity_mah # 满充容量 (mAh)
```

ROS2 CLI 示例:

代码块

```
1  ros2 topic echo /bms_state
```

4.2.2 Subscribers

4.2.2.1 /vel_cmd

类型: geometry_msgs/Twist

描述: 速度指令，用于控制机器人移动

IDL 定义:

代码块

```
1  Vector3 linear                  # 线速度
2    float64 x                    # 前进速度 (m/s)
3    float64 y                    # 未使用
4    float64 z                    # 未使用
5
6  Vector3 angular                 # 角速度
7    float64 x                    # 未使用
8    float64 y                    # 未使用
9    float64 z                    # 偏航角速度 (rad/s)
```

ROS2 CLI 示例:

代码块

```
1  # 前进0.3m/s
2  ros2 topic pub /vel_cmd geometry_msgs/Twist "{linear: {x: 0.3, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
3
4  # 原地左转0.5rad/s
5  ros2 topic pub /vel_cmd geometry_msgs/Twist "{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.5}}"
```

4.2.3 Services

4.2.3.1 /get_lowstate

类型: `lowlevel_msg/GetLowState`

描述: 按需获取底层状态（单次请求）

IDL 定义:

代码块

```
1  # Request (空)
2  ---
3  # Response
4  lowlevel_msg/LowState low_state    # 底层状态数据
5  bool success                       # 是否成功
6  string message                     # 状态消息
```

ROS2 CLI 示例:

代码块

```
1  ros2 service call /get_lowstate lowlevel_msg/srv/GetLowState
```

4.2.3.2 /head_action

类型: `lowlevel_msg/HeadAction`

描述: 头部动作控制

IDL 定义:

```

1  # Request
2  uint8 target_state           # 目标状态: 0=OFF, 1=ON
3  uint8 mode                   # 控制模式
4  bool pre_check               # 仅预检查
5  string action_path           # 动作路径
6
7  uint8 ANGLE_CONTROL = 0      # 角度控制模式
8  uint8 STREAM = 1             # 流式控制模式
9
10 int32 duration_ms            # 播放时间(ms), -1为播完
11 bool loop_enabled             # 是否循环
12 float32 playback_rate        # 播放速率 (1.0=正常)
13 float32[] target_angles      # 目标角度 [pitch, yaw] (rad)
14
15 ---
16 # Response
17 bool success                  # 是否成功
18 string message                # 状态消息
19 int32 error_code              # 错误码

```

ROS2 CLI 示例:

代码块

```

1  # 头部角度控制 (pitch=0.3, yaw=0.5)
2  ros2 service call /head_action lowlevel_msgs/srv/HeadAction "{target_state: 1,
mode: 0, target_angles: [0.3, 0.5]}"

```

4.3 双目相机 (stereo)

双目立体视觉服务, 提供H.265编码视频流和图像服务。

注意: 仅S100平台包含此服务。

4.3.1 Topics

4.3.1.1 /image_left_raw/h265和/image_right_raw/h265

类型: `foxclove_msgs/CompressedVideo`

描述: H.265编码的左/右相机视频流

还提供不同分辨率版本: `/image_left_raw/h265_half`,
`/image_left_raw/h265_quarter`, `/image_left_raw/h265_undistort`

IDL 定义:

```

1 builtin_interfaces/Time timestamp # 视频帧时间戳
2
3 string frame_id # 参考坐标系ID
4 # (原点为相机光心, +x向右, +y向下, +z向内)
5
6 uint8[] data # 压缩视频帧数据
7 # (不支持B帧; h264/h265需Annex B格式;
8 # 关键帧需包含SPS/PPS等NAL单元;
9 # 必须包含解码一帧所需的完整数据包)
10
11 string format # 视频格式
12 # (支持: "h264", "h265", "vp9", "av1")

```

ROS2 CLI 示例:

代码块

```

1 ros2 topic hz /image_left_raw/h265
2 ros2 topic echo /image_left_raw/h265 --field header

```

4.3.1.2 /stereo/left/isp_status和/stereo/right/isp_status

类型: camera_msgs/Isp2AStatus

描述: 左/右相机ISP自动曝光/白平衡状态

IDL 定义:

代码块

```

1 string camera_name # 相机名称
2 uint32 version # ISP版本
3
4 # 曝光信息
5 int32 exposure_mode # 曝光模式
6 float32 exp_time # 曝光时间
7 float32 again # 模拟增益
8 float32 dgain # 数字增益
9 float32 ispgain # ISP增益
10 float32 ae_exp # AE曝光值
11 uint32 cur_lux # 当前亮度
12 uint32 frame_id # 帧ID
13
14 # 白平衡信息
15 int32 awb_mode # AWB模式
16 float32 rgain # R增益
17 float32 grgain # GR增益

```

```

18 float32 gbgain          # GB增益
19 float32 bgain           # B增益
20
21 uint64 timestamp_ns     # 时间戳(ns)

```

ROS2 CLI 示例:

代码块

```

1 ros2 topic echo /stereo/left/isp_status
2 ros2 topic echo /stereo/right/isp_status

```

4.3.2 Services

4.3.2.1 /get_jpeg_images

类型: camera_msgs/GetJpegImages

描述: 获取JPEG格式图像

IDL 定义:

代码块

```

1  # Request
2  JpegRequest[] request:
3      uint32 channel_id      # 通道ID
4      uint32 width           # 宽度
5      uint32 height          # 高度
6      uint8 quality          # JPEG质量 (0-100)
7      bool undistort         # 是否去畸变
8
9  ---
10 # Response
11 JpegResponse[] response:
12     int32 status            # 状态码
13     uint32 width            # 实际宽度
14     uint32 height           # 实际高度
15     uint8[] data            # JPEG数据

```

ROS2 CLI 示例:

代码块

```

1 ros2 service call /get_jpeg_images camera_msgs/srv/GetJpegImages "{request:
  [{channel_id: 0, width: 640, height: 480, quality: 80, undistort: false}]}"

```

4.3.2.2 /load_stereo_calibration

类型: camera_msgs/LoadStereoCalibration

描述: 加载双目标定参数

IDL 定义:

代码块

```
1  # Request (空)
2  ---
3  # Response
4  bool success                # 是否成功
5  string message              # 消息
6  sensor_msgs/CameraInfo left # 左相机内参
7  sensor_msgs/CameraInfo right # 右相机内参
```

ROS2 CLI 示例:

代码块

```
1  ros2 service call /load_stereo_calibration
   camera_msgs/srv/LoadStereoCalibration
```

4.4 激光雷达 (lidar)

激光雷达服务, 提供点云数据和时间同步功能。

4.4.1 Topics

4.4.1.1 /lidar_points

类型: sensor_msgs/PointCloud2

描述: 激光雷达点云数据

IDL 定义:

代码块

```
1  std_msgs/Header header          # 时间戳和坐标系
2      builtin_interfaces/Time stamp # 时间戳
3      string frame_id             # 坐标系ID
4
5  geometry_msgs/Quaternion orientation # 姿态 (四元数)
6      float64 x
```

```

7    float64 y
8    float64 z
9    float64 w
10
11   float64[9] orientation_covariance          # 姿态协方差矩阵 (3x3)
12
13   geometry_msgs/Vector3 angular_velocity      # 角速度 (rad/s)
14     float64 x
15     float64 y
16     float64 z
17
18   float64[9] angular_velocity_covariance      # 角速度协方差矩阵 (3x3)
19
20   geometry_msgs/Vector3 linear_acceleration   # 线性加速度 (m/s²)
21     float64 x
22     float64 y
23     float64 z
24
25   float64[9] linear_acceleration_covariance   # 加速度协方差矩阵 (3x3)

```

ROS2 CLI 示例:

代码块

```

1  ros2 topic hz /lidar_points
2  ros2 topic echo /lidar_points --field header

```

4.4.1.2 /lidar_imu

类型: `sensor_msgs/msg/Imu`

描述: 激光雷达内置IMU数据

IDL 定义:

代码块

```

1  std_msgs/Header header          # 时间戳和坐标系
2    builtin_interfaces/Time stamp  # 时间戳
3    string frame_id              # 坐标系ID
4
5  uint8[] data                   # 原始雷达数据包 (二进制格式)

```

ROS2 CLI 示例:

代码块

```
1  ros2 topic echo /lidar_imu
```

4.4.1.3 /lidar_packets

类型: `lidar_msg/msg/LidarPacket`

描述: 激光雷达原始数据包

IDL 定义:

代码块

```
1  std_msgs/Header header          # 时间戳和坐标系
2      builtin_interfaces/Time stamp  # 时间戳
3      string frame_id             # 坐标系ID
4
5  uint8[] data                    # 原始雷达数据包（二进制格式）
```

ROS2 CLI 示例:

代码块

```
1  ros2 topic echo /lidar_packets
```

4.5 外设服务 (peripheral)

外设管理服务，包括显示屏、灯光、触摸、风扇等控制。

4.5.1 表情显示 (display/emotion)

4.5.1.1 Services

4.5.1.1.1 /display_node/play_emotion

类型: `peripheral_msgs/PlayEmotion`

描述: 播放表情动画

IDL 定义:

代码块

```
1  # Request
2  uint8 target_state          # 目标状态：0=OFF, 1=ON
3  bool pre_check              # 仅预检查
4  uint8 mode                  # 表情类型编号（通过get_supported_emotions获取）
5  int32 duration_ms           # 播放时间(ms)，-1=播完后恢复默认表情
```



```

6    ---
7    # Response
8    bool success                # 是否成功
9    string message              # 消息
10   int32 error_code            # 错误码

```

ROS2 CLI 示例:

代码块

```

1  # 播放表情 (mode=19对应019_happy)
2  ros2 service call /display_node/play_emotion peripheral_msgs/srv/PlayEmotion "
   {target_state: 1, pre_check: false, mode: 19, duration_ms: -1}"
3
4  # 停止表情
5  ros2 service call /display_node/play_emotion peripheral_msgs/srv/PlayEmotion "
   {target_state: 0, pre_check: false, mode: 0, duration_ms: 0}"

```

4.5.1.1.2 /display_node/get_supported_emotions

类型: `peripheral_msgs/GetSupportedEmotions`

描述: 获取支持的表情列表

IDL 定义:

代码块

```

1  # Request (空)
2  ---
3  # Response
4  bool success                # 是否成功
5  string message              # 消息
6  string[] supported_emotions # 支持的表情列表

```

ROS2 CLI 示例:

代码块

```

1  ros2 service call /display_node/get_supported_emotions
   peripheral_msgs/srv/GetSupportedEmotions

```

4.5.1.1.3 /display_node/backlight_ctrl

类型: `peripheral_msgs/BacklightCtrl`

描述: 背光亮度控制

IDL 定义:

代码块

```
1  # Request
2  uint8 value                # 亮度值 (0-255)
3  ---
4  # Response
5  bool success              # 是否成功
6  string message            # 消息
```

ROS2 CLI 示例:

代码块

```
1  # 设置最大亮度
2  ros2 service call /display_node/backlight_ctrl
    peripheral_msgs/srv/BacklightCtrl "{value: 255}"
3
4  # 设置50%亮度
5  ros2 service call /display_node/backlight_ctrl
    peripheral_msgs/srv/BacklightCtrl "{value: 128}"
```

4.5.1.1.4 /display_node/display_imgs

类型: peripheral_msgs/DisplayImgs

描述: 显示自定义图片序列

IDL 定义:

代码块

```
1  # Request
2  ShowImg[] images:         # 图片数组
3    uint32 width             # 宽度 (固定480)
4    uint32 height           # 高度 (固定854)
5    uint8[] data            # 图片数据
6    string encoding          # 编码格式
7    uint32[] duration_ms    # 每张图片显示时间(ms)
8  ---
9  # Response
10 bool success              # 是否成功
11 string message            # 消息
```

4.5.1.2 Topics

4.5.1.2.1 /display_info

类型: `peripheral_msgs/DisplayInfo`

描述: 显示状态信息

IDL 定义:

代码块

```
1  string[] key           # 显示信息的键数组
2  string[] info         # 显示信息的值数组
3  bool resident         # 是否为常驻显示
4  string type           # 显示类型
```

ROS2 CLI 示例:

代码块

```
1  ros2 topic echo /display_info
```

4.5.2 灯光控制 (Light)

4.5.2.1 Services

4.5.2.1.1 /light_node/control

类型: `peripheral_msgs/LightControl`

描述: 灯光控制

IDL 定义:

代码块

```
1  # Request
2  uint8 target_state      # 目标状态: 0=OFF, 1=ON
3  bool pre_check         # 仅预检查
4
5  # 模式常量
6  uint8 FIXEDCOLOR=0     # 固定颜色
7  uint8 GRADIENT=1       # 渐变
8  uint8 BREATHING=2      # 呼吸
9  uint8 FLASHING=3       # 闪烁
10 uint8 CIRCLEANDFLASH=4 # 转一圈+闪一下
11 uint8 PULSE=5          # 脉冲
```

```

12
13  uint8 mode                # 模式
14  uint8 red                 # 红色值 (0-255)
15  uint8 green              # 绿色值 (0-255)
16  uint8 blue               # 蓝色值 (0-255)
17  uint8 brightness         # 亮度 (0-255)
18  float32 speed            # 变化速度 (0.1-10.0, 1.0为默认)
19  uint16 duration_ms       # 持续时间(ms), 0=无限
20  ---
21  # Response
22  bool success              # 是否成功
23  string message            # 消息
24  int32 error_code         # 错误码

```

ROS2 CLI 示例:

代码块

```

1  # 白色常亮
2  ros2 service call /light_node/control peripheral_msgs/srv/LightControl "
   {target_state: 1, pre_check: false, mode: 0, red: 255, green: 255, blue: 255,
   brightness: 200, speed: 1.0, duration_ms: 0}"
3
4  # 红色呼吸灯
5  ros2 service call /light_node/control peripheral_msgs/srv/LightControl "
   {target_state: 1, pre_check: false, mode: 2, red: 255, green: 0, blue: 0,
   brightness: 255, speed: 0.5, duration_ms: 0}"
6
7  # 关闭灯光
8  ros2 service call /light_node/control peripheral_msgs/srv/LightControl "
   {target_state: 0, pre_check: false, mode: 0, red: 0, green: 0, blue: 0,
   brightness: 0, speed: 0, duration_ms: 0}"

```

4.5.2.1.2 /light_node/gradient

类型: `peripheral_msgs/LightGradient`

描述: 渐变灯效

IDL 定义:

代码块

```

1  # Request
2  uint8[] colors             # 颜色序列 [R1,G1,B1,R2,G2,B2,...]
3  uint8 brightness          # 亮度 (0-255)
4  float32 transition_time   # 切换时间(秒)

```

```

5   bool loop                                # 是否循环
6   uint16 duration_ms                       # 持续时间(ms), 0=无限
7   ---
8   # Response
9   bool success                             # 是否成功
10  string message                           # 消息

```

ROS2 CLI 示例:

代码块

```

1   # 红绿蓝渐变循环
2   ros2 service call /light_node/gradient peripheral_msgs/srv/LightGradient "
    {colors: [255,0,0, 0,255,0, 0,0,255], brightness: 200, transition_time: 1.0,
    loop: true, duration_ms: 0}"

```

4.5.2.1.3 /light_node/status

类型: peripheral_msgs/GetLightStatus

描述: 获取灯光状态

IDL 定义:

代码块

```

1   # Request (空)
2   ---
3   # Response
4   bool is_enabled                         # 是否启用
5   uint8 current_mode                     # 当前模式
6   uint8 current_red                     # 红色值
7   uint8 current_green                   # 绿色值
8   uint8 current_blue                    # 蓝色值
9   uint8 current_brightness              # 亮度
10  float32 current_speed                  # 速度
11  bool success                           # 是否成功
12  string message                         # 消息

```

ROS2 CLI 示例:

代码块

```

1   ros2 service call /light_node/status peripheral_msgs/srv/GetLightStatus

```

4.5.2.1.4 /light_node/fill_light

类型: `peripheral_msgs/FillLight`

描述: 补光灯控制

IDL 定义:

代码块

```
1  # Request
2  bool enable                # 启用/禁用
3  uint8 brightness          # 亮度百分比 (0-100)
4  ---
5  # Response
6  bool success               # 是否成功
7  string message             # 消息
```

ROS2 CLI 示例:

代码块

```
1  # 开启补光灯 50%亮度
2  ros2 service call /light_node/fill_light peripheral_msgs/srv/FillLight "
   {enable: true, brightness: 50}"
3
4  # 关闭补光灯
5  ros2 service call /light_node/fill_light peripheral_msgs/srv/FillLight "
   {enable: false, brightness: 0}"
```

4.5.3 触摸感应 (Touch)

4.5.3.1 Topics

4.5.3.1.1 /touch_node/touch_state

类型: `peripheral_msgs/TouchState`

描述: 触摸状态

IDL 定义:

代码块

```
1  uint8 source                # 来源: 0=S100, 1=X5
2  uint64 timestamp            # 时间戳 (ms)
3  uint8 state                 # 状态: 0=释放, 1=按下
4  uint8 idx                   # 触摸区域索引
```

ROS2 CLI 示例:

代码块

```
1  ros2 topic echo /touch_node/touch_state
```

4.5.3.2 Services

4.5.3.2.1 /touch_node/butt_enable

类型: std_srvs/SetBool

描述: 启用/禁用臀部触摸感应

IDL 定义:

代码块

```
1  bool data                # 数据（如硬件使能/失能）
2  ---
3  bool success             # 成功标志（指示服务运行成功）
4  string message           # 消息（信息提示，如错误描述）
```

ROS2 CLI 示例:

代码块

```
1  # 启用触摸感应
2  ros2 service call /touch_node/butt_enable std_srvs/srv/SetBool "{data: true}"
3
4  # 禁用触摸感应
5  ros2 service call /touch_node/butt_enable std_srvs/srv/SetBool "{data: false}"
```

4.5.4 风扇控制 (Fan)

4.5.4.1 Services

4.5.4.1.1 /fan_node/control

类型: peripheral_msgs/FanControl

描述: 风扇转速控制

IDL 定义:

代码块

```

1  # Request
2  uint8 fan_idx           # 风扇编号：0=板子风扇，1=尾部风扇
3  float32 speed_percent  # 转速百分比（0.0-100.0）
4  ---
5  # Response
6  bool success           # 是否成功
7  string message         # 消息

```

ROS2 CLI 示例:

代码块

```

1  # 板子风扇 50%转速
2  ros2 service call /fan_node/control peripheral_msgs/srv/FanControl "{fan_idx:
  0, speed_percent: 50.0}"
3
4  # 尾部风扇 100%转速
5  ros2 service call /fan_node/control peripheral_msgs/srv/FanControl "{fan_idx:
  1, speed_percent: 100.0}"
6
7  # 关闭板子风扇
8  ros2 service call /fan_node/control peripheral_msgs/srv/FanControl "{fan_idx:
  0, speed_percent: 0.0}"

```

4.5.4.1.2 /fan_node/speed

类型: `peripheral_msgs/FanSpeed`

描述: 获取风扇当前转速

IDL 定义:

代码块

```

1  # Request
2  uint8 fan_idx           # 风扇编号：0=板子风扇，1=尾部风扇
3  ---
4  # Response
5  bool success           # 是否成功
6  uint32 speed_rpm       # 转速（rpm）
7  string message         # 消息

```

ROS2 CLI 示例:

代码块


```
1 # 查询板子风扇转速
2 ros2 service call /fan_node/speed peripheral_msgs/srv/FanSpeed "{fan_idx: 0}"
```

4.5.5 GNSS定位

4.5.5.1 Topics

4.5.5.1.1 /gnss/fix

类型: sensor_msgs/NavSatFix

描述: GNSS定位数据

IDL 定义:

代码块

```
1 Header header # 消息头 (包含ROS时间戳和天线坐标系)
2
3 # 定位状态
4 NavSatStatus status # 卫星定位状态信息
5
6 # 位置数据
7 float64 latitude # 纬度 [度] (北纬为正, 南纬为负)
8 float64 longitude # 经度 [度] (东经为正, 西经为负)
9 float64 altitude # 海拔高度 [m] (WGS 84椭球之上)
10
11 # 协方差信息 (ENU: 东-北-天 坐标系)
12 float64[9] position_covariance # 位置协方差 [m^2]
13
14 # 协方差类型常量
15 uint8 COVARIANCE_TYPE_UNKNOWN = 0 # 未知
16 uint8 COVARIANCE_TYPE_APPROXIMATED = 1 # 近似值 (如基于DOP估算)
17 uint8 COVARIANCE_TYPE_DIAGONAL_KNOWN = 2 # 对角线已知 (方差已知)
18 uint8 COVARIANCE_TYPE_KNOWN = 3 # 完全已知
19
20 uint8 position_covariance_type # 位置协方差类型
```

ROS2 CLI 示例:

代码块

```
1 ros2 topic echo /gnss/fix
```

4.5.5.1.2 /signal_quality_4g

类型: `peripheral_msgs/SignalQuality`

描述: 4G信号质量

IDL 定义:

代码块

```
1  std_msgs/Header header      # 消息头
2  uint8 rssi                  # 信号强度
3  uint8 ber                    # 误码率
```

ROS2 CLI 示例:

代码块

```
1  ros2 topic echo /signal_quality_4g
```

5. 附录

5.1 通用ROS2命令

代码块

```
1  # 列出所有话题
2  ros2 topic list
3
4  # 列出所有服务
5  ros2 service list
6
7  # 列出所有节点
8  ros2 node list
9
10 # 查看话题信息
11 ros2 topic info /rt/lowstate
12
13 # 查看服务类型
14 ros2 service type /display_node/play_emotion
15
16 # 查看消息定义
17 ros2 interface show lowlevel_msg/msg/LowState
```

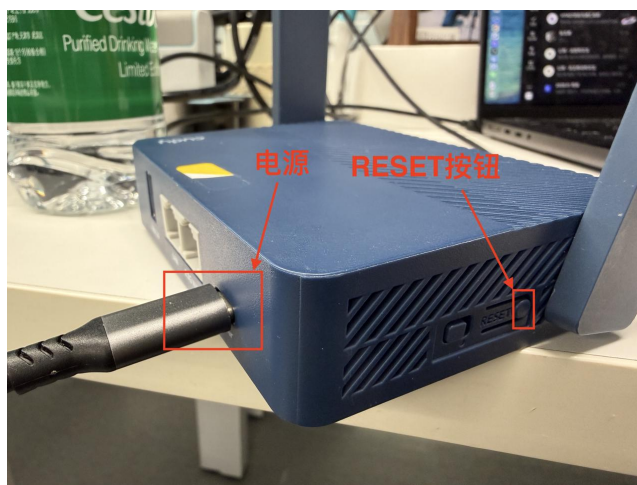
5.2 环境变量

代码块 RMW实现 (默认zenoh)

```
2 export RMW_IMPLEMENTATION=rmw_zenoh_cpp
3
4 # ROS2 Domain ID
5 export ROS_DOMAIN_ID=0
```

5.3 便携式Wi-Fi路由器

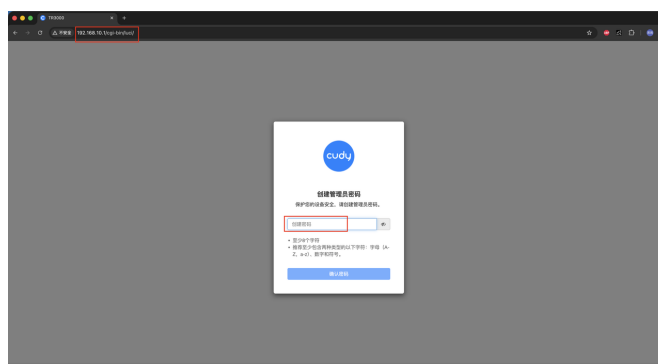
首先，给路由器插上电（第一次使用无需RESET）：



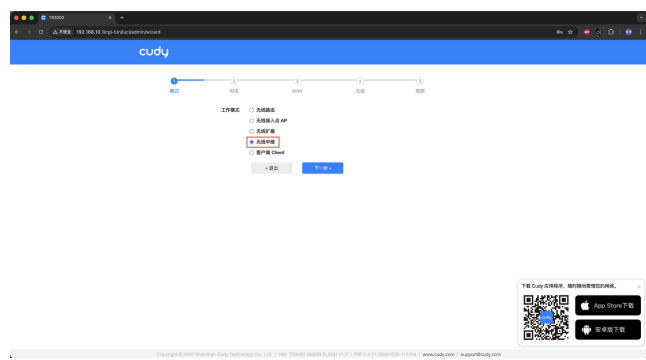
然后从Wi-Fi列表中找到对应的名称（例如Cudy-089F），并连接：



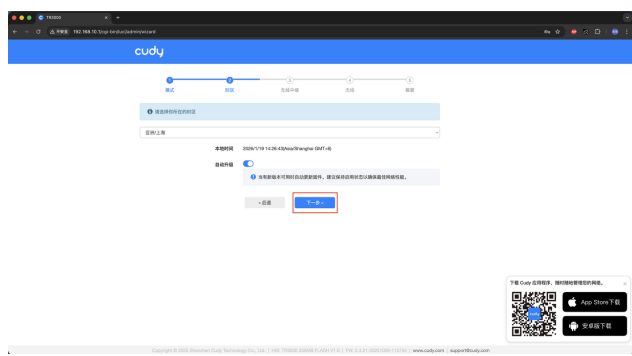
然后网页打开192.168.10.1（管理地址），并创建管理员密码（需要自行记住），再点击「确认密码」：



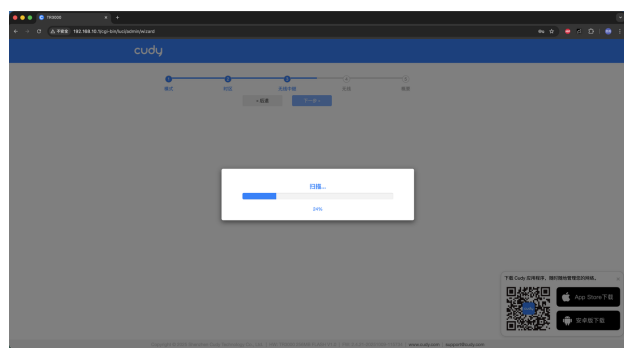
然后点击「无线中继」，并点击「下一步」：



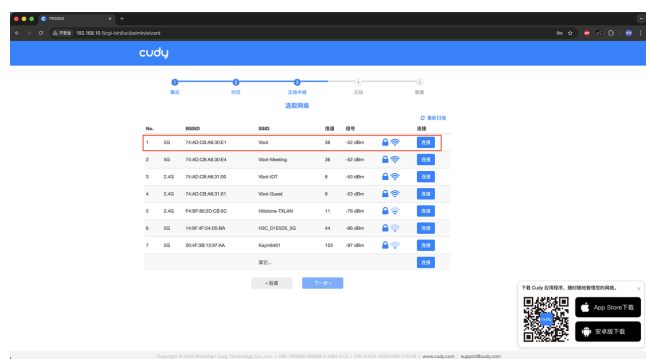
点击「下一步」：



开启扫描：



选择想连接的Wi-Fi，点击「连接」和「下一步」：



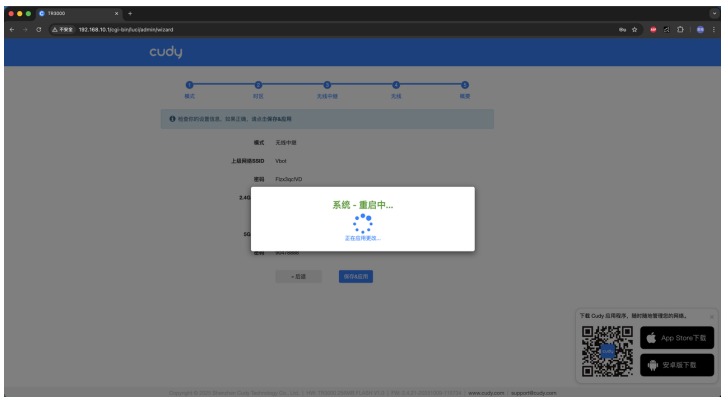
点击「下一步」：



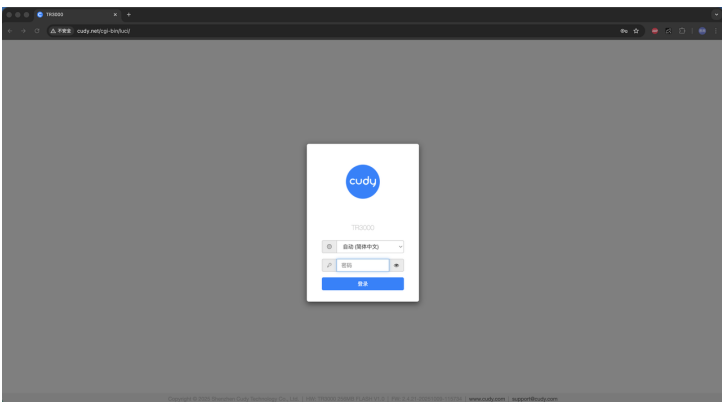
点击「保存&应用」：



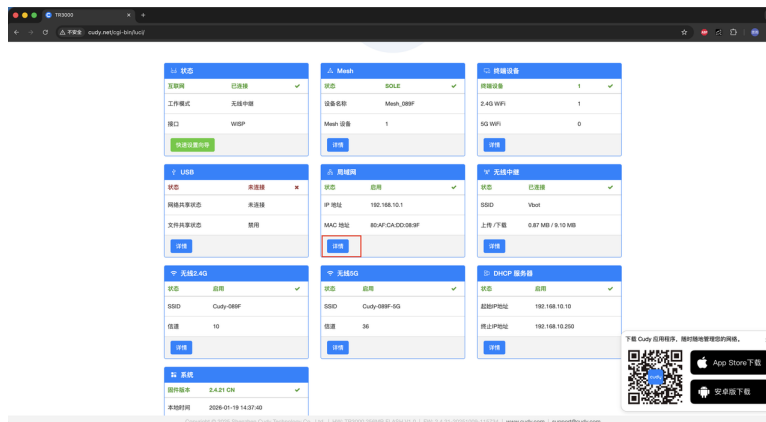
系统重启，耐心等待一会，中间会切网，需要自行再连回该Wi-Fi：



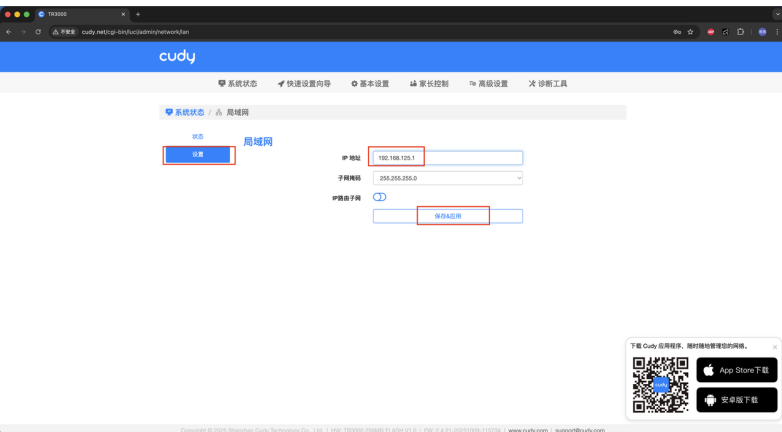
连回该Wi-Fi后，还是登陆管理员账号：



点击「局域网」的「详情」：



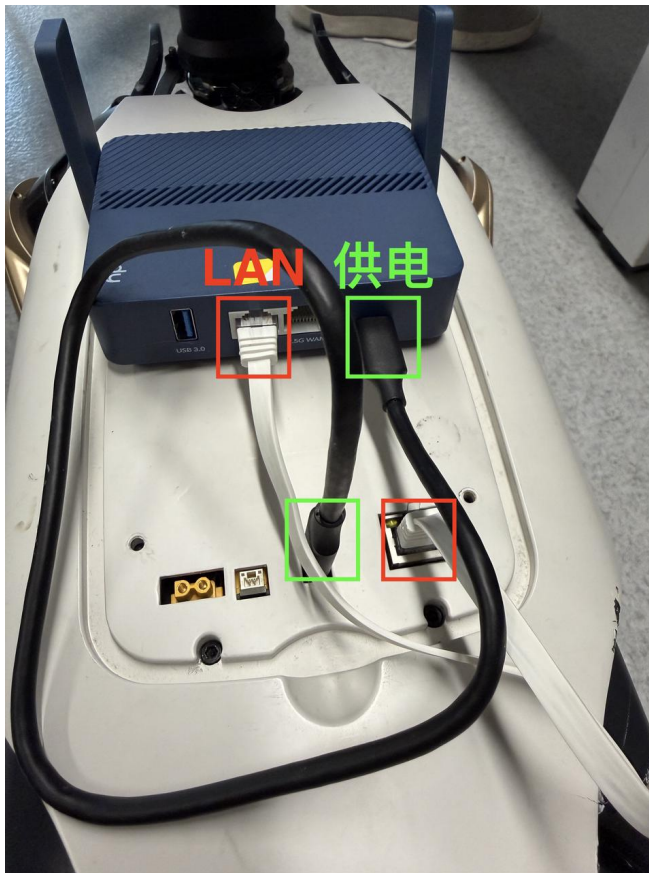
点击「设置」，将IP地址配置为192.168.125.1，再点击「保存&应用」：



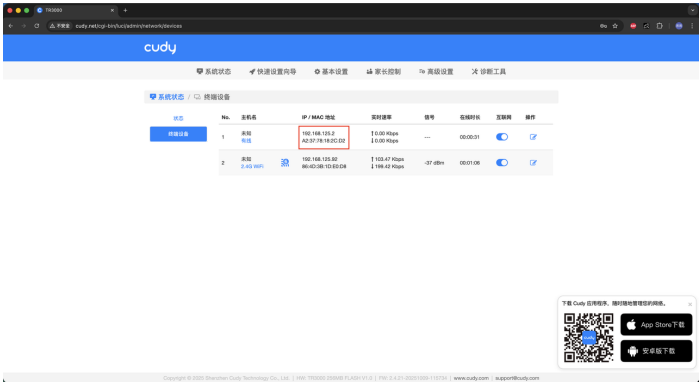
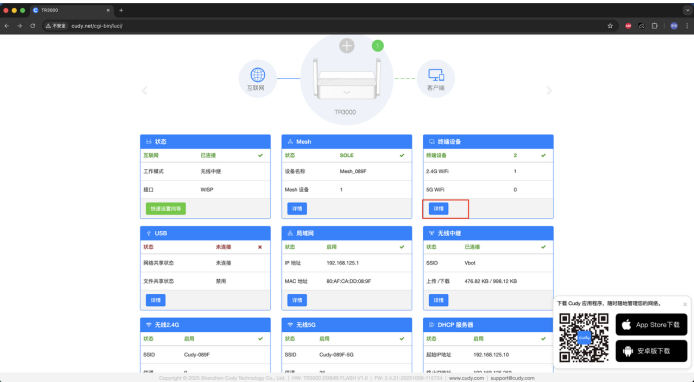
系统重启，耐心等待一会儿：



将路由器和狗连接：



PC连回该Wi-Fi，并进入192.168.10.1，点击「终端设备」的「详情」，便能看到vbot设备（192.168.125.2）：



由此，便能通过局域网连接vbot。